Python Program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


# Construct expression tree from prefix expression
def construct_tree(prefix):
    stack = []

    for ch in reversed(prefix):
        node = Node(ch)

        if ch in "+-*/":
            if len(stack) < 2:
                raise ValueError("Invalid prefix expression")

            node.left = stack.pop()
            node.right = stack.pop()

        stack.append(node)

    if len(stack) != 1:
        raise ValueError("Invalid prefix expression")

    return stack.pop()


# Non-recursive post-order traversal
def postorder(root):
    if root is None:
        return

    stack1 = [root]
    stack2 = []

    while stack1:
        node = stack1.pop()
        stack2.append(node)

        if node.left:
            stack1.append(node.left)

        if node.right:
            stack1.append(node.right)

    print("Post-order traversal:", end=" ")

    while stack2:
        print(stack2.pop().data, end=" ")

    print()
```

```
# Delete the entire tree
def delete_tree(root):
    if root is None:
        return

    stack = [root]

    while stack:
        node = stack.pop()

        if node.left:
            stack.append(node.left)

        if node.right:
            stack.append(node.right)

        node.left = None
        node.right = None
        node.data = None

    print("Tree deleted successfully.")


# Main program
prefix = input("Enter prefix expression: ")

try:
    root = construct_tree(prefix)
    postorder(root)
    delete_tree(root)
    root = None

except ValueError as e:
    print(e)
```

Sample Input and Output

Input prefix expression: +a*bc

```
Enter prefix expression: +a*bc
Post-order traversal: a b c * +
Tree deleted successfully.
```